

Plan de Pruebas

“SAPE C.S.S”

Fecha:[28/09/2026]

Tabla de contenido

Histórico de Revisiones

Versión	Fecha	Descripción/cambio	autor
1.0	28/09/2026	Desarrollo	Felipe Catalan

Información del Proyecto

Organización	Duoc UC
Sección	001D
Proyecto (Nombre)	SAPE CSS
Fecha de Inicio	
Fecha de Término	
Patrocinador principal	Duoc Uc
Docente	JUAN ANDRES HERNANDEZ ALIAGA

Integrantes

Rut	Nombre	Correo

Alcance de las pruebas

Definición de requisitos de S.W., módulos de Software a probar, Requisitos ambiente de pruebas y Documentación Referenciada, etc.

El proceso de pruebas del sistema **SAPE CSS** abarca la verificación y validación funcional, estructural y de seguridad de todos los módulos que conforman el Producto Mínimo Viable (MVP).

1. Módulos y requisitos de software dentro del alcance:

- **Módulo de Identidad, Autenticación y Autorización:** Pruebas del flujo de inicio de sesión, cierre de sesión, recuperación de contraseña y asignación estricta de los 3 roles internos del sistema mediante Spatie Permission (*Administrador / Directivo, Profesor Jefe y Profesor de Asignatura*). Verificación de que el perfil *Apoderado* es un stakeholder externo sin usuario, sin inicio de sesión y sin permisos en el sistema.
- **Módulo Generador de Datos Sintéticos:** Pruebas del comando generador de datos ficticios en formato oficial réplica SIGE (NOMINA y CALIFICACIONES), verificando la integridad de cabeceras, catálogos curriculares reales (1° a 4° Medio) y casos de borde forzados (estudiantes retirados, cuellos de botella en Matemáticas, notas conceptuales en Religión y eximidos en Educación Física).
- **Módulo de Ingesta y Procesamiento ETL:** Pruebas de carga masiva de archivos Excel, validación estructural de columnas, detección de duplicados, atomicidad e idempotencia transaccional (reversión total o *rollback* ante fallos en cualquier fila para no publicar datos parciales corruptos).
- **Módulo de Analítica y Motor de Reglas Decreto 67:** Pruebas de cálculo de promedios, porcentajes de asistencia, detección de materias cuello de botella y generación de indicadores explicables de riesgo académico según la normativa del Ministerio de Educación de Chile (Decreto 67).
- **Módulo de Asistente IA y Privacidad por Diseño:** Pruebas del mecanismo de seudonimización y tokenización de datos previo a cualquier llamada externa a la API de Inteligencia Artificial (Google Gemini), procesamiento asíncrono mediante colas de trabajo con respuesta visual en ≤ 2 segundos y validación de que todo resultado generado sea siempre un borrador pedagógico sujeto a revisión humana.
- **Módulo de Comunicación Asistida (WhatsApp wa.me):** Pruebas de generación dinámica de enlaces seguros *wa.me* para citación de apoderados y registro en bitácora de auditoría en base de datos (se prueba la generación del enlace y su trazabilidad, no el envío ni la entrega efectiva del mensaje).
- **Módulo de Frontend y Dashboards:** Pruebas de navegación, renderizado reactivo de mapas térmicos, gráficos de rendimiento y tablas dinámicas según el rol del usuario conectado.

2. Elementos fuera del alcance:

- Modelos predictivos avanzados basados en Machine Learning (scikit-learn / Python), formalmente catalogados como experimento futuro posterior al MVP.
- Integración directa con APIs transaccionales pagadas de Meta o servicios automatizados de mensajería masiva (se utiliza la función estándar y gratuita Click to Chat de WhatsApp).
- Acceso o portal web para apoderados (no contemplado por diseño para resguardar la superficie de seguridad).
- Pruebas de estrés masivo a gran escala (el universo de usuarios reales del colegio es de aproximadamente 40 docentes y directivos).

3. Requisitos del ambiente de pruebas:

- Entorno local totalmente reproducido en contenedores Docker con PostgreSQL 16 y base de datos de pruebas dedicada `sape_test`.
- Uso exclusivo de datos sintéticos generados algorítmicamente para dar estricto cumplimiento a la Ley 19.628 y preparar la Ley 21.719 de protección de datos de menores de edad.

4. Documentación referenciada:

- *Especificación de Proyecto de Título (EPT v1.5).*
- *Documento de Arquitectura de Sistema (DAS v1.2).*
- *Stack Tecnológico v1.3.*
- *Formato de Datos SIGE v1.0* (Documentos md/Formato_Datos_SIGE.md).
- *Decisiones de Arquitectura INDG-16 e INDG-26* (docs/decisiones/)

Descripción

Descripción general del sistema y características clave

SAPE CSS (Sistema de Alerta Preventiva Escolar) es una plataforma web desarrollada para el Colegio San Sebastián de Melipilla cuyo propósito principal es transformar los registros históricos de notas y asistencia en alertas tempranas de riesgo de deserción o reprobación, facilitando intervenciones pedagógicas oportunas antes del cierre del año escolar.

Características clave del sistema:

1. Monolito Modular Eficiente: Desarrollado sobre Laravel 12 en el backend e Inertia.js con React y Tailwind CSS en el frontend, garantizando una navegación fluida como aplicación de página única (SPA) sin la sobrecarga de mantener APIs REST separadas.
2. Motor de Diagnóstico Basado en Normativa Real: Evalúa el desempeño estudiantil bajo los criterios transparentes y auditables del Decreto 67 del Mineduc (combinación de promedios críticos y porcentaje de asistencia mínima), descartando cálculos de "caja negra" incomprensibles para el cuerpo docente.
3. Privacidad por Diseño y Protección de Menores: El software no expone identidades reales de estudiantes a servicios en la nube. Cuenta con un sistema interno de tokenización previa a las consultas de IA y un generador propio de datos sintéticos para desarrollo, pruebas y demostraciones.
4. Asistente Pedagógico y Directivo con IA: Integra capacidades de asistencia pedagógica para apoyar a los docentes en la redacción de pautas, rúbricas y estrategias de refuerzo escolar, operando de manera asíncrona para no congelar la pantalla del usuario.
5. Comunicación Asistida a Costo Cero: Genera enlaces directos de WhatsApp prellenados con el contexto de la alerta para que el profesor jefe cite personalmente al apoderado con un solo clic, sin costos de mensajería empresarial y registrando cada citación en una bitácora institucional.

--

Resumen de las pruebas	
Módulos del sistema a probar	Módulo de Autenticación y Perfiles: Registro, inicio de sesión, reseteo de claves y autorización de rutas según rol Spatie. Módulo Generador de Datos Sintéticos: Creación de planillas Excel réplica SIGE con notas, asistencia y casos de borde. Módulo de Ingesta y Procesamiento ETL: Subida, validación de estructura, carga atómica a base de datos y reversión por error. Módulo de Analítica y Motor Decreto 67: Cálculo de riesgo escolar, promedios de curso, semestrales y asignaturas cuello de botella. Módulo de Comunicación WhatsApp (wa.me): Construcción de URLs, precarga de mensaje y registro de auditoría en bitácora. Módulo Asistente IA y Tokenizador: Seudonimización de nombres de estudiantes, cola asíncrona y validación de esquemas de respuesta. Módulo Frontend y Dashboards: Interfaz visual en React, filtros por curso, despliegue de mapas térmicos y tablas de estudiantes.
Objetivos de las pruebas	- Validar que la plataforma cumpla al 100% los requerimientos funcionales y criterios de aceptación acordados con el colegio patrocinador. - Garantizar la integridad referencial y consistencia de los datos académicos ante cualquier intento de carga con errores. Comprobar que ningún dato real de estudiantes sea expuesto o enviado a proveedores externos de Inteligencia Artificial. - Verificar que las operaciones habituales en la interfaz respondan en un tiempo no superior a 2 segundos en condiciones de uso normales.

	• Asegurar la estabilidad del sistema mediante una suite de pruebas automatizadas que prevenga la aparición de errores en funcionalidades ya existentes.
Detalle del orden de ejecución de los módulos	La ejecución de las pruebas sigue un orden de dependencias de menor a mayor nivel de integración: 1. Fase Seguridad y Cimientos: Pruebas del módulo de autenticación, control de sesiones y protección de rutas por roles. 2. Fase Fuente de Datos: Pruebas del generador sintético para asegurar que las planillas de prueba cumplan la estructura exacta del Mineduc. 3. Fase Procesamiento ETL: Pruebas de ingesta sobre la base de datos, validando detección de inconsistencias y rollback transaccional. 4. Fase Reglas de Negocio: Pruebas del motor de indicadores Decreto 67 sobre los datos cargados en la base de datos. 5. Fase Servicios y Conectores: Pruebas de tokenización, cliente de IA encolado y generación de enlaces de citación wa.me. 6. Fase Experiencia de Usuario y Visualización: Pruebas de integración del frontend React con los datos entregados por los controladores Inertia.
Tipos de pruebas a realizar	• Pruebas Unitarias: Validación aislada de algoritmos de cálculo, generador de RUT/DV ficticio, reglas de validación y transformadores de datos. • Pruebas de Integración (Feature Tests): Ejecución de flujos completos contra una base de datos PostgreSQL real aislada (sape_test), comprobando transacciones, inserciones y respuestas HTTP. • Pruebas de Regresión: Ejecución automática de la suite completa en

	cada cambio de código para asegurar que no se introduzcan fallas colaterales. • Pruebas Funcionales de Interfaz y Usabilidad: Pruebas manuales estructuradas simulando los tres roles reales (Director, Profesor Jefe y Profesor de Asignatura). • Pruebas de Seguridad y Permisos: Intentos deliberados de acceso a rutas directivas desde cuentas de profesores y verificación de que usuarios sin sesión sean redirigidos al login.								
Técnicas de pruebas a utilizar	• Pruebas de Caja Negra: Diseñadas en base a los Criterios de Aceptación de cada Historia de Usuario, evaluando entradas y salidas sin depender de la implementación interna. • Pruebas de Caja Blanca: Pruebas de código que analizan caminos lógicos, control de excepciones, transacciones de base de datos y métodos internos de los servicios. • Análisis de Valores Límite y Clases de Equivalencia: Pruebas centradas en notas de borde (ej. 3.9 reprobatorio vs. 4.0 aprobatorio) y porcentajes de asistencia límite (ej. 84.9% en riesgo vs. 85.0% regular). • Pruebas Deterministas con Semillas (Seeds): Uso de valores de inicialización fijos para que la generación de datos de prueba sea 100% reproducible en cualquier equipo. • Análisis Estático de Código: Chequeo estricto de estándares y convenciones de código mediante Laravel Pint antes de admitir cualquier cambio.								
Roles y responsabilidades	<table> <tr> <th>Rol</th> <th>Responsabilidades</th> </tr> <tr> <td>Felipe Catalán</td> <td>.</td> </tr> <tr> <td>José Valenzuela</td> <td></td> </tr> <tr> <td>Felipe Verdugo</td> <td></td> </tr> </table>	Rol	Responsabilidades	Felipe Catalán	.	José Valenzuela		Felipe Verdugo	
Rol	Responsabilidades								
Felipe Catalán	.								
José Valenzuela									
Felipe Verdugo									

Entorno y configuración de las pruebas

Definir los requisitos de software y hardware necesarios para ejecutar las pruebas.

Para garantizar que las pruebas se ejecuten de manera confiable y reproduzcan exactamente el comportamiento del sistema en producción, el equipo utiliza un entorno estandarizado e idéntico en las máquinas de todos los integrantes.

1. Requisitos de Hardware:

- **Procesador:** CPU de 4 núcleos o superior (x86_64).
- **Memoria RAM:** Mínimo 8 GB (16 GB recomendado para la ejecución simultánea de contenedores y compilación de frontend).
- **Almacenamiento:** Mínimo 20 GB de espacio libre en disco de estado sólido (SSD).
- **Conectividad:** Conexión a internet para la descarga de paquetes y resolución de llamadas de prueba a la API de IA.

2. Requisitos de Software:

- **Sistema Operativo:** Windows 10/11 con WSL2 habilitado o Linux Ubuntu 22.04 / 24.04 LTS.
- **Motor de Base de Datos:** PostgreSQL 16 ejecutado en contenedor Docker oficial (sistema_escolar_db).
- **Runtime Backend:** PHP 8.3 con extensiones obligatorias activas: pdo_pgsql, pgsql, fileinfo, zip, intl y gd.
- **Gestor de Dependencias Backend:** Composer 2.7+.
- **Runtime Frontend:** Node.js versión 22.x y gestor npm 10.x.
- **Control de Versiones y Virtualización:** Git 2.40+ y Docker Desktop 4.30+ con soporte para Docker Compose v2.

3. Configuración de la Base de Datos de Pruebas:

- Para asegurar paridad absoluta con el servidor final y el entorno de integración continua, las pruebas **no utilizan SQLite en memoria**, sino una base de datos PostgreSQL real y aislada llamada sape_test.
- La configuración de conexión se encuentra predefinida en el archivo de configuración phpunit.xml: host 127.0.0.1, puerto 5432, base de datos sape_test, usuario admin_db y contraseña correspondiente.

4. Instrucciones estándar de ejecución (orden estricto):

```
```bash
```

Continuación...Más instrucciones dentro del repositorio.

**Calendarización de las actividades de pruebas**

*Listado de actividades, tareas, duración, fechas, responsables, etc.*

Adjuntar carta Gantt

### Resumen de riesgos

Listado de riesgos relacionado al proceso de pruebas de S.W. Indicar riesgo, magnitud o impacto de este riesgo por etapa en el proceso. Magnitud: Alto, Significativo, Moderado, Inferior y Baja. Probabilidad de ocurrencia. Plan de mitigación y plan de contingencia.

Riesgo	Magnitud	Ocurrencia	Plan de Mitigación	Plan de Contingencia
Indisponibilidad o fallo de conexión con el contenedor Docker de PostgreSQL local durante las pruebas				
Incompatibilidad o inconsistencias de formato en los archivos Excel réplica SIGE proporcionados				
Agotamiento de cuota o latencia excesiva en la API gratuita de Google Gemini durante pruebas de integración				
Exposición accidental de datos sensibles o reales de menores de edad en el ambiente de pruebas.				
Tiempo insuficiente del equipo para escribir pruebas automatizadas ante avances acelerados de desarrollo.				
Discrepancia en las reglas de cálculo del Decreto 67 entre la lógica del sistema y el criterio del colegio.				
Fallo en la suite de pruebas automatizadas por ausencia de				

compilación previa de los recursos de frontend.				
-------------------------------------------------	--	--	--	--

Definición de artefactos <i>Listar y describir los artefactos que serán administrados y entregados durante este proceso de prueba.</i>	
Artefacto	Descripción
Plan de Pruebas de Software (este documento)	Documento formal que define la estrategia, alcance, ambiente, herramientas, riesgos y criterios de aceptación para el aseguramiento de calidad del proyecto SAPE CSS.
Matriz de Casos de Prueba y Criterios de Aceptación	Registro detallado de casos de prueba derivados de las Historias de Usuario (formato Dado/Cuando/Entonces), trazables a cada requerimiento funcional del sistema.
Suite de Pruebas Automatizadas (Código)	Conjunto de archivos de prueba escritos en PHPUnit ubicados en las carpetas `tests/Unit/` y `tests/Feature/` del repositorio, ejecutables localmente y en el pipeline de integración.
Reportes de Integración Continua (CI Logs)	Registros de ejecución generados automáticamente por GitHub Actions en cada cambio o propuesta de integración a la rama `dev`, evidenciando el estado verde de estilo y pruebas.
Matriz de Archivos de Datos Sintéticos	Planillas Excel oficiales generadas en `storage/app/sinteticos/` que contienen historiales escolares completos y reproducibles con semillas deterministas para alimentar las pruebas.
Bitácora de Defectos e Incidentes (Issue Tracker)	Tablero de seguimiento de incidentes gestionado a través de GitHub Issues, donde se clasifican los errores detectados por severidad, pasos de reproducción y estado de resolución.
Acta de Conformidad y Cierre de Pruebas	Informe final emitido al término del Sprint 5 que certifica que los criterios de aceptación fueron validados satisfactoriamente para la entrega del proyecto de título.

#### Condiciones de aceptación para cierre del proceso de pruebas

*Condiciones que se deben cumplir para dar termino al proceso de pruebas y margen de tolerancia de aceptación de defectos.*

**Directriz de plantilla:** Condiciones que se deben cumplir para dar término al proceso de pruebas y margen de tolerancia de aceptación de defectos.

El proceso de pruebas se dará por concluido exitosamente y habilitará la entrega formal del software cuando se cumplan a cabalidad las siguientes condiciones objetivas:

#### 11.1 Condiciones obligatorias de cierre

- **Suite 100% en verde:** El 100% de las pruebas automatizadas de la suite del proyecto deben finalizar con resultado exitoso (**en verde**) tanto en el entorno local como en el pipeline de GitHub Actions.
- **Criterios de Aceptación validados:** El 100% de los Criterios de Aceptación definidos para las Historias de Usuario del MVP deben encontrarse verificados y aprobados mediante pruebas funcionales.
- **Rollback y atomicidad comprobada:** El mecanismo de importación masiva (ETL) debe demostrar reversión total (*rollback*) ante planillas corruptas sin generar registros huérfanos en la base de datos.
- **Privacidad y tokenización verificada:** La anonimización y tokenización previa a las consultas de Inteligencia Artificial debe estar completamente verificada, asegurando que ningún dato real de estudiantes salga del servidor.
- **Estilo de código aprobado:** El estilo de código de todo el repositorio debe superar la validación estática de Laravel Pint sin advertencias pendientes.

## 11.2 Margen de tolerancia de defectos aceptable

Para declarar apta la versión final del sistema, los defectos detectados deberán ajustarse a la siguiente escala de tolerancia:

- **Defectos Críticos (Bloqueo del sistema, caída de base de datos, brechas de seguridad o pérdida de datos):**  
**Tolerancia: 0 defectos permitidos.** Cualquier falla crítica impide el cierre y exige corrección inmediata.
- **Defectos Mayores (Fallo en un flujo funcional clave sin alternativa operativa viable):**  
**Tolerancia: 0 defectos permitidos.** Todo flujo principal (ingesta, cálculo de riesgo, asignación de roles, citación wa.me) debe operar de manera íntegra.
- **Defectos Menores / Moderados (Inconsistencias secundarias que cuentan con una alternativa de uso evidente):**  
**Tolerancia: Máximo 2 defectos permitidos.** Deben encontrarse debidamente documentados en el backlog y no comprometer la experiencia pedagógica central.
- **Defectos Cosméticos / Visuales (Desajustes menores de alineación visual o textos que no alteran la comprensión):**  
**Tolerancia: Máximo 5 observaciones permitidas.** Serán programadas para ajustes en ciclos posteriores de mantención.

### Glosario

- **API (Interfaz de Programación de Aplicaciones):** Conjunto de reglas y especificaciones que permiten a dos aplicaciones comunicarse entre sí (en el proyecto, la conexión con Google Gemini).

- **Atomicidad:** Propiedad transaccional que asegura que un conjunto de operaciones en la base de datos se ejecute en su totalidad o no se aplique ninguna, evitando datos a medio procesar.
- **Click to Chat (wa.me):** Característica pública de WhatsApp que permite iniciar una conversación sin agregar el contacto a la agenda, abriendo un chat con un texto precargado desde una URL estándar.
- **Decreto 67 (Mineduc):** Normativa chilena que regula la evaluación, calificación y promoción escolar, estableciendo criterios objetivos para determinar cuándo un alumno requiere acompañamiento pedagógico intensivo.
- **Datos Sintéticos:** Datos generados artificialmente mediante algoritmos informáticos que mantienen las mismas propiedades estadísticas y de formato que los datos reales, pero sin contener información de personas reales.
- **ETL (Extracción, Transformación y Carga):** Proceso que toma datos desde una fuente externa (archivos Excel réplica SIGE), los limpia y valida según reglas de negocio, y los almacena de forma definitiva en PostgreSQL.
- **Faker:** Biblioteca de software utilizada para generar masivamente datos ficticios creíbles (nombres, fechas, notas, direcciones) durante el desarrollo y las pruebas.
- **Idempotencia:** Propiedad por la cual una operación (por ejemplo, subir la misma planilla dos veces consecutivas) produce exactamente el mismo resultado sin duplicar registros ni corromper información.
- **Inertia.js:** Tecnología que actúa como puente entre el backend en Laravel y los componentes visuales en React, permitiendo desarrollar una aplicación moderna sin requerir una API REST separada.
- **Monolito Modular:** Enfoque arquitectónico donde todo el sistema se desarrolla dentro de una misma base de código, pero estructurado en módulos o paquetes claramente delimitados con responsabilidades específicas.
- **Pint (Laravel Pint):** Herramienta de análisis estático que asegura que el código PHP cumpla con las guías de estilo oficiales, manteniendo un código limpio y legible entre todos los miembros del equipo.
- **Pruebas de Regresión:** Pruebas que se vuelven a ejecutar tras modificar el sistema para certificar que las nuevas funciones no hayan estropeado características que ya funcionaban bien.
- **Rollback:** Operación que deshace los cambios realizados durante una transacción de base de datos en caso de que ocurra algún error imprevisto, devolviendo el sistema a su estado original seguro.
- **SIGE:** Sistema de Información General de Estudiantes del Ministerio de Educación de Chile, cuyo formato oficial de exportación de nóminas y calificaciones es replicado por el sistema SAPE CSS.
- **Spatie Permission:** Paquete ampliamente utilizado en Laravel para gestionar permisos y perfiles de usuario, garantizando que cada rol acceda exclusivamente a las pantallas autorizadas.
- **Tokenización / Seudonimización:** Técnica de seguridad que sustituye los datos personales identificables (como el nombre o RUT de un estudiante) por identificadores artificiales o códigos anónimos antes de procesarlos en servicios externos.